Hierarchical Bayesian Extension for Distributional GPT Detection

1 Background: Pooled Model (Liang et al. 2025)

We observe a corpus of sentences $\{s_1, \ldots, s_N\}$ and wish to estimate the fraction $\alpha \in [0, 1]$ that were generated by an LLM. Each sentence is modelled as a bag of words drawn from a mixture of two Bernoulli product distributions:

$$P(s \mid \alpha) = (1 - \alpha)\, P_{\text{human}}(s) + \alpha\, Q_{\text{AI}}(s), \tag{1}$$

where, for a vocabulary $\mathcal{V}$,

$$\log P(s) = \sum_{w \in \mathcal{V}} \big[\mathbb{1}[w \in s]\ \log P(w) + \mathbb{1}[w \notin s]\ \log(1 - P(w))\big], \tag{2}$$

and analogously for Q. Word-level probabilities $P(w)$ and $Q(w)$ are estimated as binary occurrence frequencies over human (pre-ChatGPT) and AI-rewritten corpora respectively, pooled across all agencies.

The mixture fraction is estimated by maximum likelihood:

$$\hat{\alpha} = \arg\max_{\alpha} \sum_{i=1}^{N} \log\big[(1 - \alpha)\, P(s_i) + \alpha\, Q(s_i)\big]. \tag{3}$$

Problem. A single pooled P cannot capture the vocabulary variation across federal agencies. Agencies with distinctive terminology (e.g. military jargon at DOD, environmental vocabulary at EPA) may have word frequencies that diverge from the pool in directions that overlap with Q, producing spurious $\hat{\alpha} > 0$ even on text known to be entirely human-written.

2 Hierarchical Bayesian Extension

We introduce agency-specific human distributions $P_a(w)$ via a conjugate Beta prior centred on the pooled estimate.

2.1 Model

Level 1: Global hyperparameters. Let $\mu_w = P_{\text{pool}}(w)$ be the pooled human frequency for word w. Let $\kappa > 0$ be a scalar concentration parameter controlling how much agencies can deviate from the pool.

Level 2: Agency-specific word probabilities. For each agency a and word w:

$$P_a(w) \sim \mathrm{Beta}\big(\kappa\,\mu_w,\ \kappa\,(1-\mu_w)\big). \tag{4}$$

This prior has mean μ_w and "effective sample size" κ. Large κ shrinks agencies toward the pool; small κ lets them diverge.

Level 3: Mixture fraction. For each stratum (agency a, time period t):

$$\alpha_{a,t} \sim \mathrm{Beta}(a_0, b_0), \tag{5}$$

with, e.g., $a_0 = 1, b_0 = 1$ (uniform) or a weakly informative prior.

Observation model. The sentence likelihood under agency a replaces P with P_a in (1)–(2), while Q remains pooled.

2.2 Graphical Model

Figure 1 shows the plate diagram for the hierarchical model.

2.3 Posterior for $P_a(w)$ (Conjugate Update)

Because the human and AI corpora are *labelled* (we know which documents are human-written and which are AI-rewritten), we can estimate $P_a(w)$ in closed form.

Let n_a be the number of human sentences from agency a, and $n_{a,w}$ the number of those sentences containing word w. The posterior is:

$$P_a(w) \mid \text{data} \ \sim\ \mathrm{Beta}\big(\kappa\,\mu_w + n_{a,w},\ \ \kappa\,(1-\mu_w) + n_a - n_{a,w}\big). \tag{6}$$

The posterior mean (our point estimate) is the **shrinkage estimator**:

$$\boxed{\hat{P}_a(w) = \frac{\kappa\,\mu_w + n_{a,w}}{\kappa + n_a}\,.} \tag{7}$$

When $n_a \gg \kappa$, the agency's own data dominates; when $n_a \ll \kappa$, $\hat{P}_a(w) \approx \mu_w$ (the pool).

2.4 Estimating κ (Empirical Bayes)

We choose κ to maximise the marginal likelihood, integrating out $P_a(w)$ analytically. Each (a, w) pair contributes a Beta-Binomial term:

$$\log p(\text{data} \mid \kappa) = \sum_a \sum_{w \in \mathcal{V}} \Big[\log B\big(\kappa\mu_w + n_{a,w},\ \kappa(1-\mu_w) + n_a - n_{a,w}\big) - \log B\big(\kappa\mu_w,\ \kappa(1-\mu_w)\big)\Big], \tag{8}$$

where $B(\cdot,\cdot)$ is the Beta function. This is a one-dimensional optimisation over κ (or equivalently $\log\kappa$) and is solved via bounded scalar minimisation.

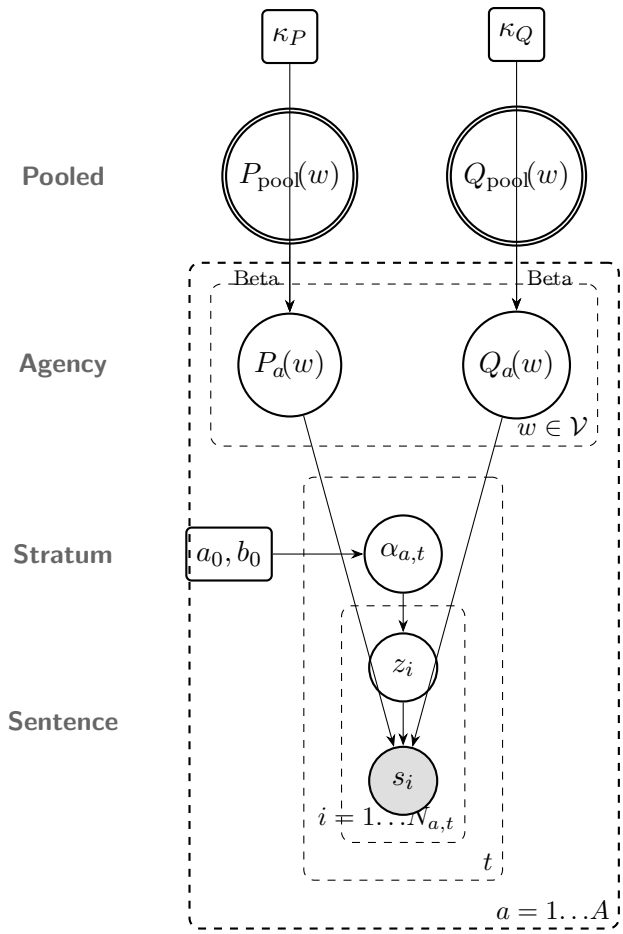

Figure 1: Plate diagram for the hierarchical distributional GPT detection model. Double-circled nodes ($P_{\text{pool}}, Q_{\text{pool}}$) are estimated from pooled corpora and held fixed. Open circles are latent variables; the shaded circle (s_i) is observed. Agency-specific word probabilities $P_a(w), Q_a(w)$ are drawn from Beta priors centred on the pooled estimates, with concentration κ_P, κ_Q estimated via empirical Bayes. Each sentence is generated from either P_a or Q_a according to the latent indicator z_i, whose Bernoulli parameter $\alpha_{a,t}$ is the AI usage fraction for stratum (a, t).

2.5 Two-Stage Estimation Procedure

1. **Estimate step.** Compute pooled $P_{\text{pool}}(w)$ and $Q(w)$ as before. Additionally, collect per-agency word counts $(n_{a,w}, n_a)$. Optimise κ via (8).

2. **Infer step.** For each stratum (a, t):

   (a) Compute $\hat{P}_a(w)$ using (7) with the estimated κ.

   (b) Replace P_{pool} with $\hat{P}_a$ in the Bernoulli model (2). Q remains pooled.

   (c) Estimate $\hat{\alpha}_{a,t}$ via MLE (3) as before.

This adds no iterative inference (no MCMC, no EM). The only new computation is the conjugate update (7) and the scalar optimisation of κ.

3 Evaluation: κ Selection via Spurious Alpha

To choose κ without overfitting, we evaluate on pre-ChatGPT data where the ground-truth $\alpha = 0$. For a grid of κ values $\{\kappa_j\}$:

1. For each agency a, compute $\hat{P}_a^{(\kappa_j)}(w)$ using (7).
2. Run MLE inference on agency a's pre-ChatGPT sentences to obtain the **spurious alpha** $\hat{\alpha}_a^{(\kappa_j)}$.
3. Report: mean, median, and maximum spurious alpha across agencies; fraction of agencies with $\hat{\alpha} > 1\%$.

The optimal κ minimises an aggregate spurious-alpha metric (e.g., mean or maximum across agencies), subject to the constraint that the model retains sensitivity to real AI-written text.

Interpretation.

- $\kappa \to \infty$: equivalent to the original pooled model; agencies with distinctive vocabulary remain vulnerable to false positives.
- $\kappa \to 0$: each agency gets its own empirical P; may overfit for small agencies (few sentences $\Rightarrow$ noisy word frequencies).
- Intermediate κ: balances pool regularisation with agency-specific adaptation. The marginal-likelihood estimator (8) provides a principled data-driven choice.

4 Implementation

The hierarchical extension is implemented in `scripts/corpus_ai_usage.py` via three flags:

```
# Step 2 (with hierarchical):
python scripts/corpus_ai_usage.py estimate --hierarchical

# Step 3 (with hierarchical):
python scripts/corpus_ai_usage.py infer --hierarchical [--kappa K]

# Evaluation sweep:
python scripts/corpus_ai_usage.py evaluate \
    --kappa-min 10 --kappa-max 100000 --kappa-steps 15
```

The `estimate --hierarchical` step saves per-agency word counts and the optimal κ alongside the standard pooled distributions. The `infer --hierarchical` step loads these and constructs $\hat{P}_a(w)$ per agency. The `evaluate` subcommand sweeps κ on pre-ChatGPT data and reports spurious alpha diagnostics. All original (pooled) code paths remain unchanged when the `--hierarchical` flag is omitted.